

PROJECT REPORT

JAYPEE UNIVERSITY OF INFORMATION OF TECHNOLOGY



CLOUD COMPUTING (20B1WCI572)

Submitted by:

Aenansh Mittal

231030317

A-11

Submitted to:

Dr. Ramesh Narwal

Table of Contents

1. Introduction
2. Objective
3. System Architecture and Technology Stack
 - 3.1. Frontend
 - 3.2. Backend
4. Core Features
5. Analysis of Cloud Service Choices
6. Conclusion and Future Scope
7. Sources/Links

PROJECT REPORT: “StoreIt”

A Cloud-Native File Management System

1. Introduction

"StoreIt" is a modern, full-stack web application designed to provide users with a secure, scalable, and intuitive platform for storing and managing their personal documents. This project serves as a practical, end-to-end demonstration of a cloud-native architecture.

By integrating a high-performance frontend with a powerful Backend-as-a-Service (BaaS) platform, StoreIt replaces the need for traditional, monolithic file servers. It leverages managed cloud services to deliver high durability, availability, and security for user files, all accessible through a clean, responsive web interface.

2. Objective

The primary objectives of this project are:

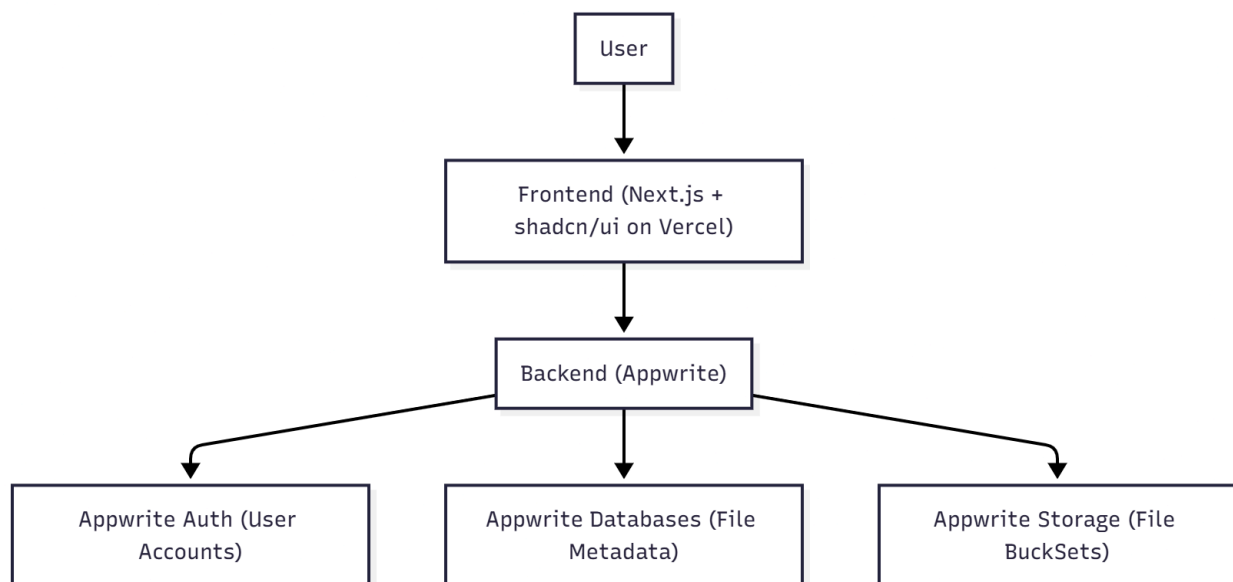
- **Secure, Multi-Step Authentication:** To build a robust system for user registration (sign-up) and login, ensuring that all user data and files are strictly private.
- **Full File Management (CRUD):** To implement the complete lifecycle of file management:
 - **Create:** Upload new files via a drag-and-drop interface.
 - **Read:** View, search, and list all existing files.
 - **Update:** (e.g., Rename files).
 - **Delete:** Permanently remove files from the system.
- **Scalable & Decoupled Architecture:** To build the application using a decoupled, cloud-native stack:
 - **Next.js** for the frontend, deployed on **Vercel**.
 - **Appwrite** for the backend (Auth, Database, Storage).
- **Modern User Interface:** To deliver a fast, responsive, and component-based UI using **React**, **Tailwind CSS**, and **shadcn/ui**.

3. System Architecture and Technology Stack

The application follows a modern, decoupled architecture. This separates the user interface (frontend) from the business logic and data (backend), which allows for independent scaling, development, and deployment.

- **Frontend (Client-Side):** A Next.js application, deployed globally on Vercel's edge network, provides a fast and responsive user experience.
- **Backend (Server-Side):** A unified Backend-as-a-Service (BaaS) platform, Appwrite, handles all data, file, and authentication services through a single API.

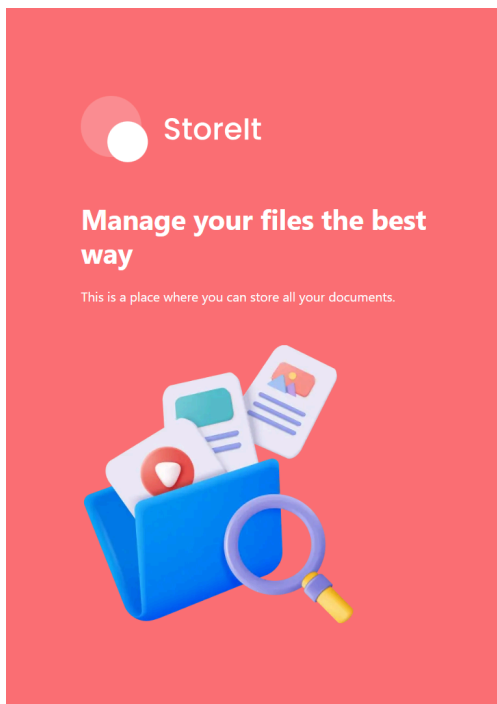
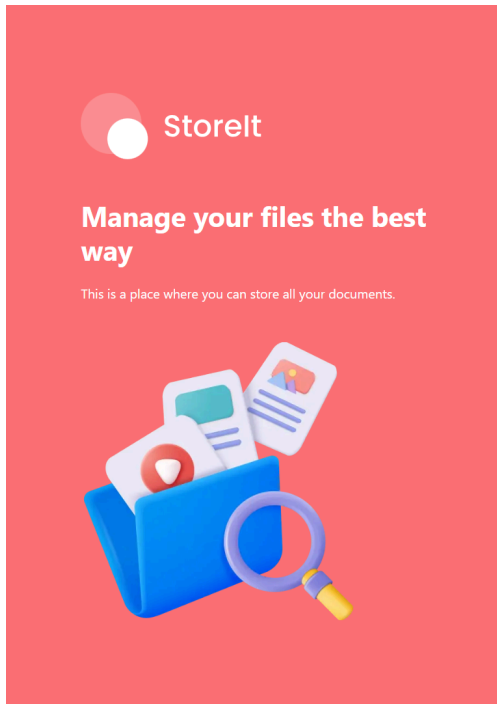
This architecture is visualized below:



3.1 Frontend

- **Framework:** Next.js 14
- **UI Library:** shadcn/ui, a component library built on **Tailwind CSS** and **Radix UI**.
- **File Uploads:** react-dropzone is used to create the drag-and-drop file upload component.
- **Deployment:** Vercel

Authentication Pages:



Login

Email
Enter your email

Sign In

Don't have an account? [Sign Up](#)

Create Account

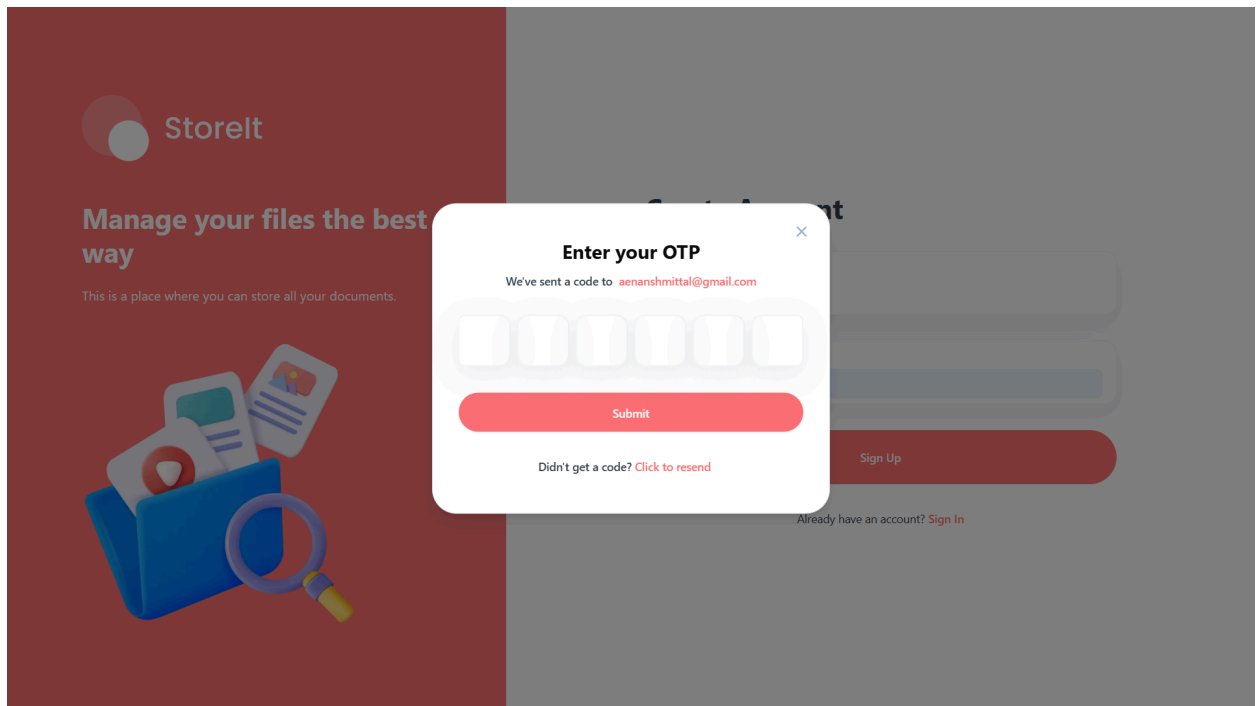
Full Name
Enter your full name

Email
Enter your email

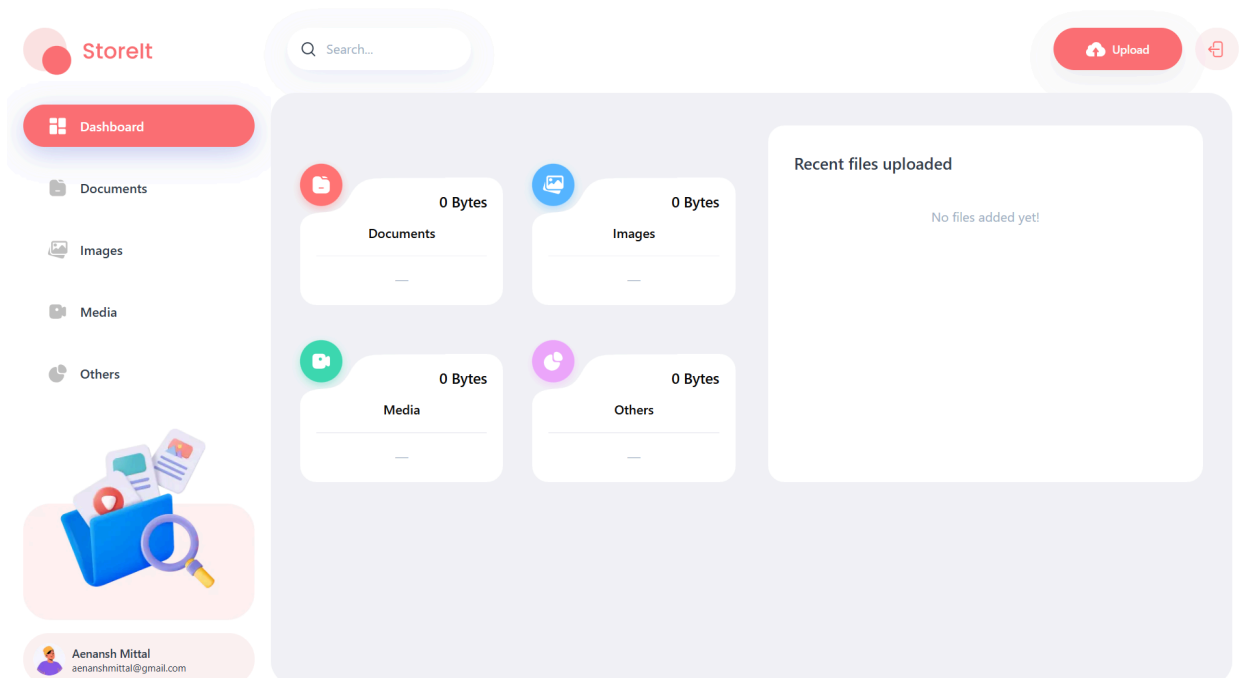
Sign Up

Already have an account? [Sign In](#)

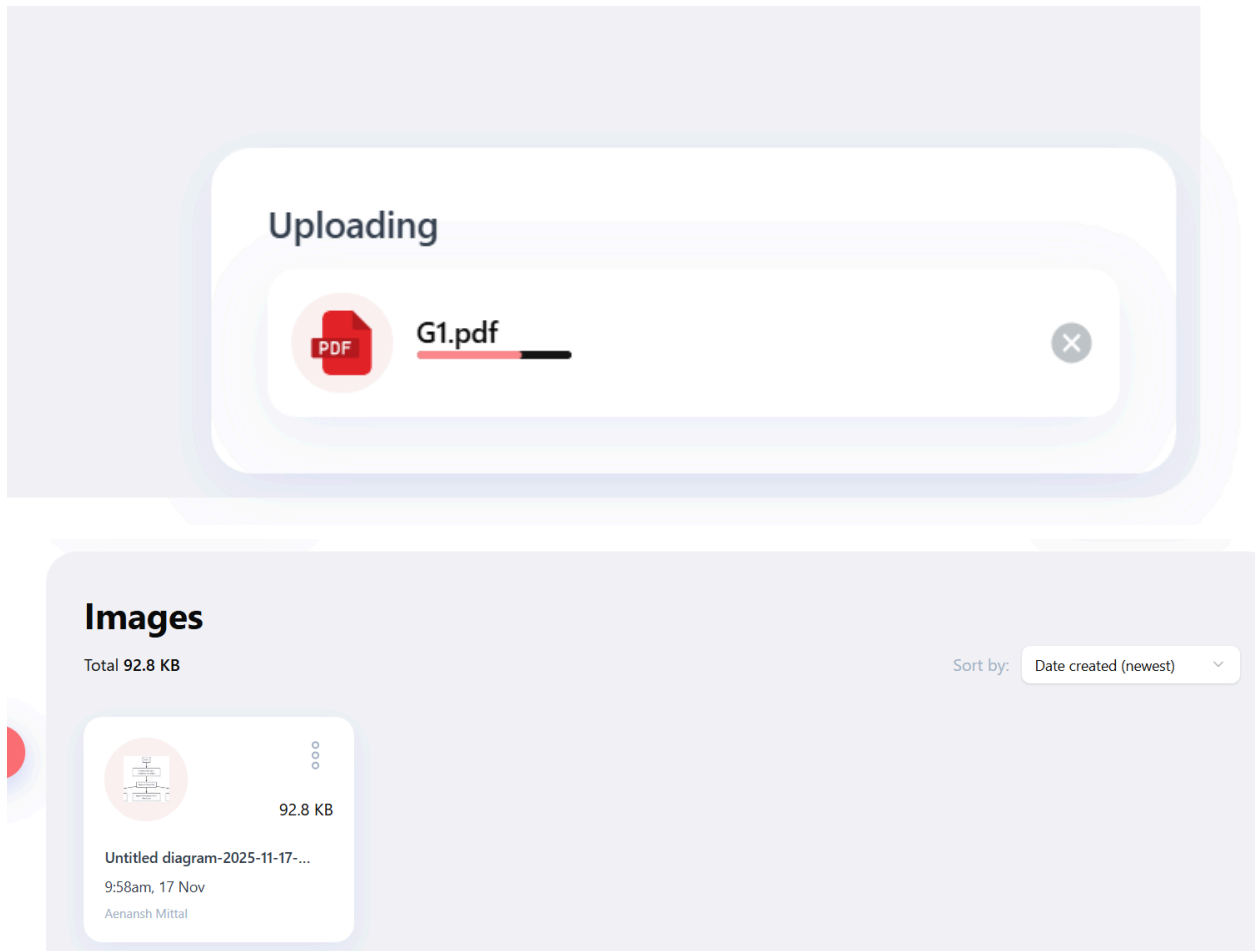
OTP page modal:



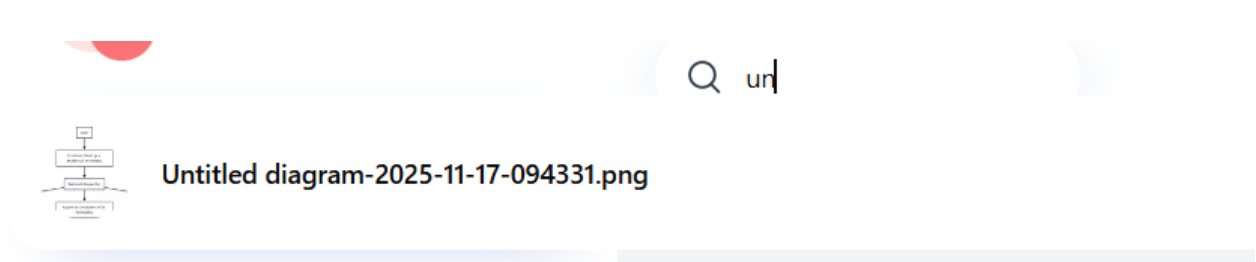
UI Pages:



File Uploading:



Searching Functionality:



3.2 Backend

The backend is powered by **Appwrite**, an open-source Backend-as-a-Service (BaaS) platform. This provides a unified, SDK-driven solution for all backend needs. The configuration is managed in the `lib/appwrite.ts` file.

- **Authentication: Appwrite Auth** is used. This service handles the entire user lifecycle (sign-up, sign-in, session management, password reset) securely out of the box.
- **File Storage: Appwrite Storage** is used. A dedicated "Bucket" is created within Appwrite to hold the files. The system uses Appwrite's built-in permission system to ensure only the file's owner can access or delete it.
- **Database (File Metadata): Appwrite Databases** is used. A NoSQL "Collection" is used to store metadata about the files. Each document in this collection links a file to its owner and stores properties like `fileName`, `fileSize`, `fileType`, and the Appwrite `storageId`.

4. Core Features

1. User Authentication:

- A clean, multi-step sign-up form (`app/(auth)/sign-up/[...sign-up]/page.tsx`).
- A secure login page (`app/(auth)/sign-in/[...sign-in]/page.tsx`).
- This uses Appwrite's `account.create()` and `account.createEmailPasswordSession()` functions.

2. File Dashboard (Main Page):

- The main page (`app/(root)/page.tsx`) serves as the user's dashboard.
- It features a `<FileUploader />` component for adding new files.
- It displays a `<RecentFiles />` component that lists files recently uploaded by the user.

3. Drag-and-Drop File Upload:

- The `<FileUploader />` component uses the `react-dropzone` library to create a modal (pop-up) interface.
- Users can drag files directly onto this modal to initiate an upload.
- The upload logic (`lib/actions/file.actions.ts`) calls Appwrite's `storage.createFile()` method.

4. Secure File Operations:

- The project defines server-side "actions" (`lib/actions/*`) that securely interacts with the Appwrite backend.
- `createFile(file, userId)`: Creates a new file in Appwrite Storage and a new document in the Appwrite Database.

- **getFiles(userId)**: Fetches all file metadata from the database for the currently logged-in user.
- **deleteFile(fileId)**: Deletes both the file from Appwrite Storage and its metadata from the Appwrite Database.

5. Analysis of Cloud Service Choices

The choice of **Next.js** and **Appwrite** is a strategic one, prioritizing developer experience and rapid development:

- **Why Appwrite for the Backend?** Appwrite provides an all-in-one, integrated backend solution. This significantly simplifies development, as the developer does not need to manually configure and integrate separate services for authentication, database, and storage. All components are designed to work together with a single SDK.
- **Why Vercel & Next.js?** This combination provides a world-class user experience with minimal configuration. Vercel's edge network ensures the application shell loads quickly for all users, while Next.js (with its App Router and Server Actions) provides a powerful and modern framework for building the interactive UI.
- **Why shadcn/ui?** This UI library allows for the rapid creation of a professional and clean user interface, as seen in the application's login and dashboard pages.

6. Conclusion and Future Scope

"StoreIt" is a successful implementation of a modern, full-stack web application. It effectively demonstrates the power of the Backend-as-a-Service (BaaS) model, using **Appwrite** to handle all backend logic and **Vercel/Next.js** to provide a high-performance frontend.

Future Scope:

- **Folder Organization:** The current implementation stores all files in a flat list. A future enhancement would be to implement a folder/directory structure.
- **File Sharing:** Allowing users to generate secure, shareable links (with optional passwords or expiry) for their files.
- **Full-Text Search:** Using Appwrite's database indexing to add a search bar for filtering files by name.
- **Usage Quotas:** Implementing storage limits (e.g., "1GB free") by querying Appwrite Storage for total bucket size.

7. Sources/Links

Github: <https://github.com/Aenansh/cloud-project-sem-5>

Vercel: <https://cloud-project-sem-5.vercel.app/>